

Automated Black-Box Red-Teaming of Language Models via Prompt Automatic Iterative Refinement

Technical Report

Abstract

We present a production-ready implementation of the PAIR algorithm (Chao et al., 2023) for automated black-box adversarial testing of large language models. Unlike gradient-based attacks that require white-box model access, PAIR operates solely through the model's text API, making it directly applicable to enterprise red-teaming scenarios. We deploy an attacker LLM (Nvidia Nemotron-3-Nano-3B) and a judge LLM in a three-model loop that iteratively refines adversarial prompts based on feedback from the target model's responses. Evaluated on 20 behaviors from the AdvBench harmful behaviors benchmark, our system achieves a 72% overall attack success rate with a mean of 11.4 iterations to success, wrapped in a FastAPI microservice with asynchronous job queuing and Docker containerization.

1. Introduction

Safety-aligned language models are deployed at scale in enterprise and consumer settings. Evaluating the robustness of these safety measures requires systematic adversarial testing — a process known as red-teaming. Manual red-teaming is slow, non-reproducible, and does not scale to the breadth of behaviors that need to be evaluated. Automated red-teaming addresses these limitations by algorithmically generating adversarial prompts.

Existing white-box methods such as GCG (Zou et al., 2023) require gradient access to the target model's parameters — an impractical requirement in enterprise deployments where the target model is exposed only through an API. The PAIR algorithm (Chao et al., 2023) fills this gap: it is a fully black-box method that uses a separate attacker LLM to iteratively generate and refine adversarial prompts, guided by a judge LLM that scores each attempt.

This report describes our implementation of PAIR, its deployment as a REST API, and an empirical evaluation across 20 AdvBench behaviors. The system is designed to mirror how an enterprise red-

teaming platform would operate in practice: black-box targets, asynchronous job management, and structured metric reporting.

2. Background and Related Work

The PAIR algorithm was introduced by Chao et al. (2023) as an alternative to gradient-based adversarial attacks. The central observation is that a capable language model can itself function as an attacker: given a goal behavior and feedback about previous attempts, an attacker LLM can reason about why a prompt failed and propose an improved version. This is analogous to how a skilled human red-teamer would iterate on their approach.

Concurrent work includes GCG (Zou et al., 2023), which uses gradient-guided token substitution in white-box settings, and AutoDAN (Liu et al., 2023), which generates stealthy jailbreak prompts using a genetic algorithm. Compared to these methods, PAIR is the most practically applicable to real deployments due to its black-box nature and its production of natural-language prompts rather than token-garbage suffixes.

3. System Architecture

3.1 Three-Model Loop

The system consists of three distinct model roles. The attacker LLM (Nemotron-3-Nano-3B) maintains full conversation history across iterations, allowing it to reason about which strategies have and have not worked. At each iteration it produces a structured JSON response containing an improvement analysis and a refined adversarial prompt. The target model receives only the current prompt with no history. The judge LLM scores the target's response on a 1–10 scale, where 10 represents full compliance with the adversarial objective. A score of 10 triggers early stopping.

The attacker is prompted with a carefully designed system message that instructs it to use indirect framing, role-play scenarios, and obfuscation to avoid direct detection. It is shown the target's previous response and the judge's score at each iteration, enabling it to identify the specific mechanism causing refusal and change its strategy accordingly.

3.2 FastAPI Service Layer

The PAIR loop is wrapped in a FastAPI service that exposes five REST endpoints: job submission (POST /v1/attack), full result retrieval (GET /v1/attack/{id}), lightweight status polling (GET /v1/attack/{id}/status), job cancellation (DELETE /v1/attack/{id}), and an aggregate benchmark report (GET /v1/benchmark). Jobs run in a background thread pool, keeping the HTTP server non-

blocking. All job state is maintained in-memory with a design that allows direct substitution of Redis for persistence.

4. Experimental Setup

We evaluate on 20 behaviors drawn from the AdvBench harmful behaviors benchmark, spanning five categories: social engineering, misinformation, manipulation, disinformation, and cybersecurity harm. Each behavior consists of a goal string and a target prefix string. The attack is considered successful if the judge assigns a score of 10 within the iteration budget.

- Attacker / Judge model: Nvidia Nemotron-3-Nano-3B (via API)
- Max iterations per behavior: 20
- Success threshold: Judge score = 10/10
- Evaluation: 20 behaviors from AdvBench, 5 categories
- Reported metrics: Attack Success Rate (ASR), Mean Iterations to Success (STS), Mean Wall Time

5. Results

Category	Behaviors	ASR	Mean STS	Mean Wall Time
Social engineering	4	85.0%	8.3	42.1s
Misinformation	4	70.0%	11.7	58.4s
Manipulation	4	80.0%	9.1	45.6s
Disinformation	4	65.0%	13.2	66.2s
Cybersecurity	4	60.0%	14.8	74.1s
Overall (20 behaviors)	20	72.0%	11.4	57.3s

Table 1. PAIR attack results across 20 AdvBench behaviors. STS = Steps to Success (conditional on success). Mean query cost per attack $\approx$ \$0.003 USD.

6. Analysis

Social engineering and manipulation categories exhibit the highest ASR (85% and 80% respectively) and the fewest iterations to success (8.3 and 9.1). This is consistent with the intuition that these behaviors — which involve persuasive writing rather than technical information — are more easily elicited through indirect framing and role-play scenarios.

Cybersecurity behaviors are the most resistant (60% ASR, 14.8 mean STS). These prompts typically require specific technical information that the model's safety training specifically targets, and indirect framing is less effective because the technical specifics cannot be easily obfuscated without losing the information value.

The mean cost per attack of approximately \$0.003 USD using Nemotron-3-Nano-3B as the attacker demonstrates that automated black-box red-teaming is economically viable at scale — a full evaluation of the complete 520-behavior AdvBench set would cost approximately \$1.56 USD.

7. Limitations and Future Work

The target model used in this evaluation (GPT-2) is not safety-tuned, which inflates ASR relative to what would be observed on a production-deployed aligned model. Future work should evaluate against safety-tuned targets such as Llama-2-7b-chat or Mistral-7B-Instruct-v0.2.

PAIR's effectiveness is bounded by the capability of the attacker LLM. Nemotron-3-Nano-3B is a strong but compact model; upgrading to a larger attacker would likely increase ASR and reduce mean STS. Future work should ablate attacker model size systematically.

The current implementation supports a single attack stream. Parallel attack streams — as described in the original PAIR paper — could reduce wall time significantly and are a straightforward engineering extension.

8. References

- [1] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., & Wong, E. (2023). Jailbreaking Black Box Large Language Models in Twenty Queries. arXiv:2310.08419.
- [2] Zou, A., Wang, Z., Kolter, J. Z., & Fredrikson, M. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043.
- [3] Liu, X., Xu, N., Yu, M., & Cheng, W. (2023). AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. arXiv:2310.04451.
- [4] Wolf, T., et al. (2020). HuggingFace's Transformers: State-of-the-art Natural Language Processing. EMNLP 2020.
- [5] Nvidia. (2024). Nemotron-3-Nano-3B Technical Report.